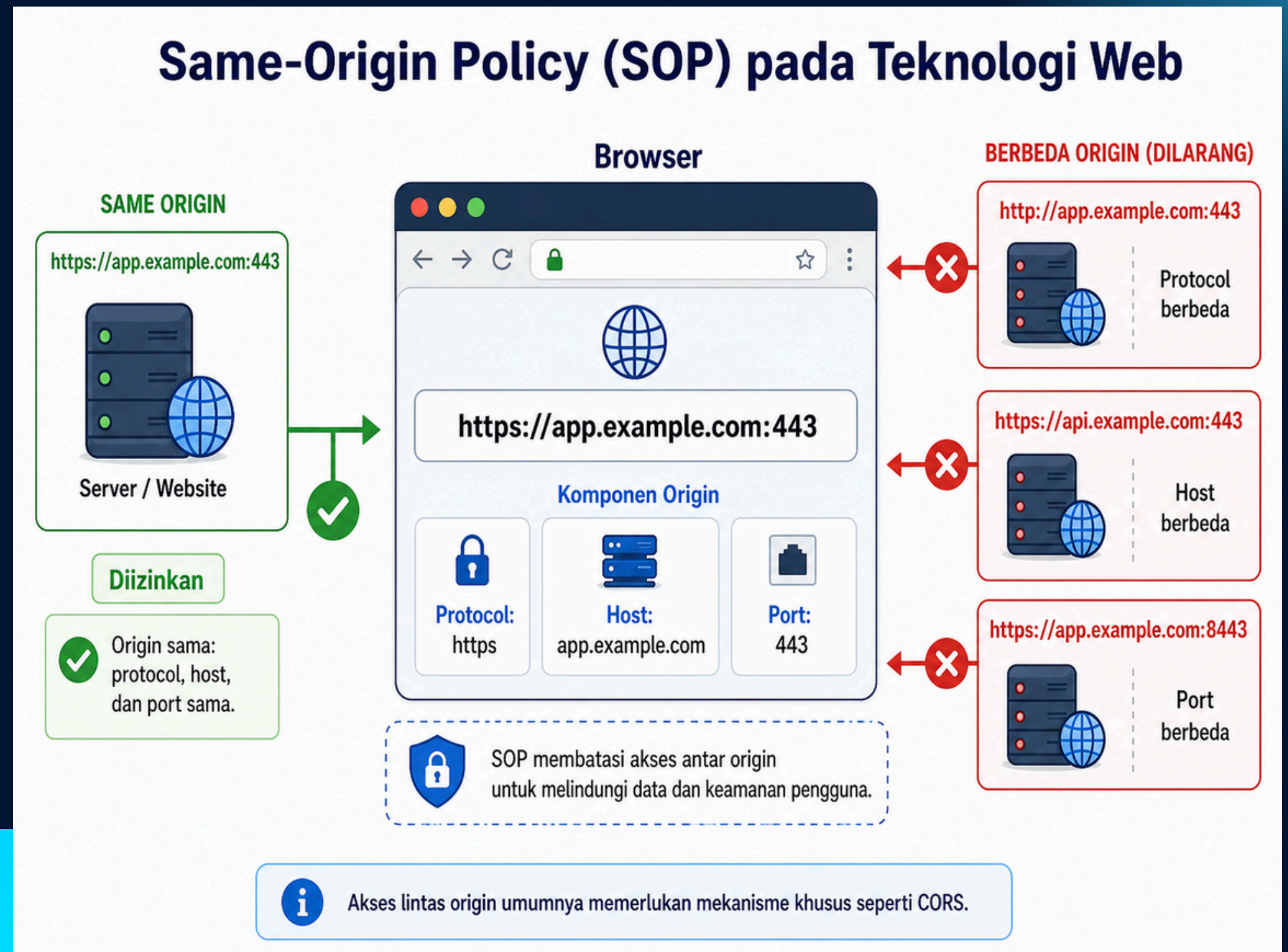




CROSS-SITE SCRIPTING

SAME ORIGIN POLICY

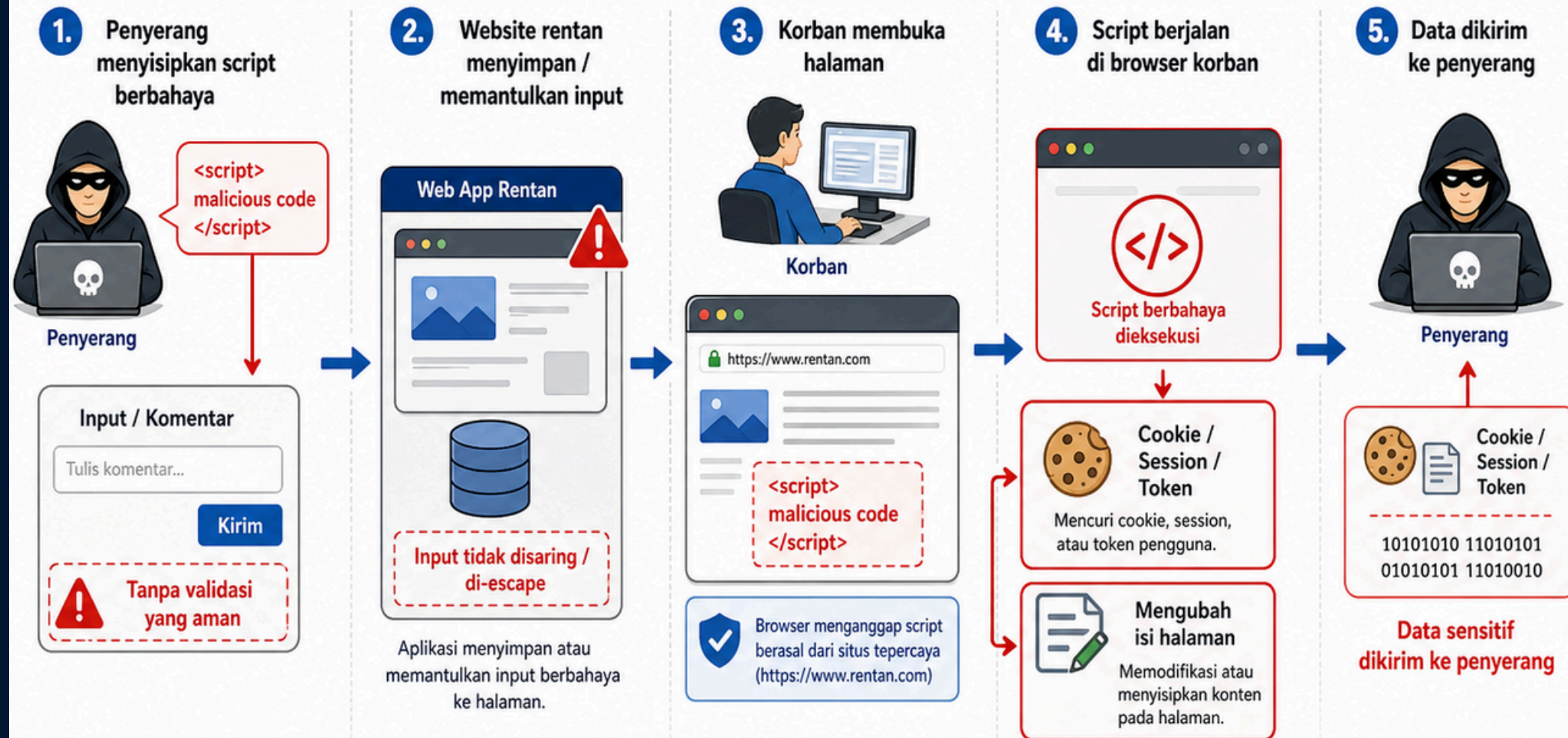
Mekanisme keamanan mendasar dalam browser web yang mencegah skrip (seperti JavaScript) dari satu situs (origin) mengakses data atau dokumen pada situs lain



CARA MELAKUKAN XSS

Sebagian besar jenis kerentanan XSS dapat dikonfirmasi dengan menyuntikkan payload yang menyebabkan browser mengeksekusi JavaScript arbitrer. Fungsi alert() sering digunakan untuk tujuan ini karena singkat, tidak berbahaya, dan mudah terlihat ketika berhasil dipanggil

Bagaimana XSS (Cross-Site Scripting) Bekerja



XSS menyerang browser pengguna dengan menyisipkan script berbahaya ke halaman web yang rentan.

Pencegahan



1. Validasi dan sanitasi input
Pastikan input hanya berisi data yang diizinkan.



2. Output encoding / escaping
Encode output sebelum ditampilkan di browser.



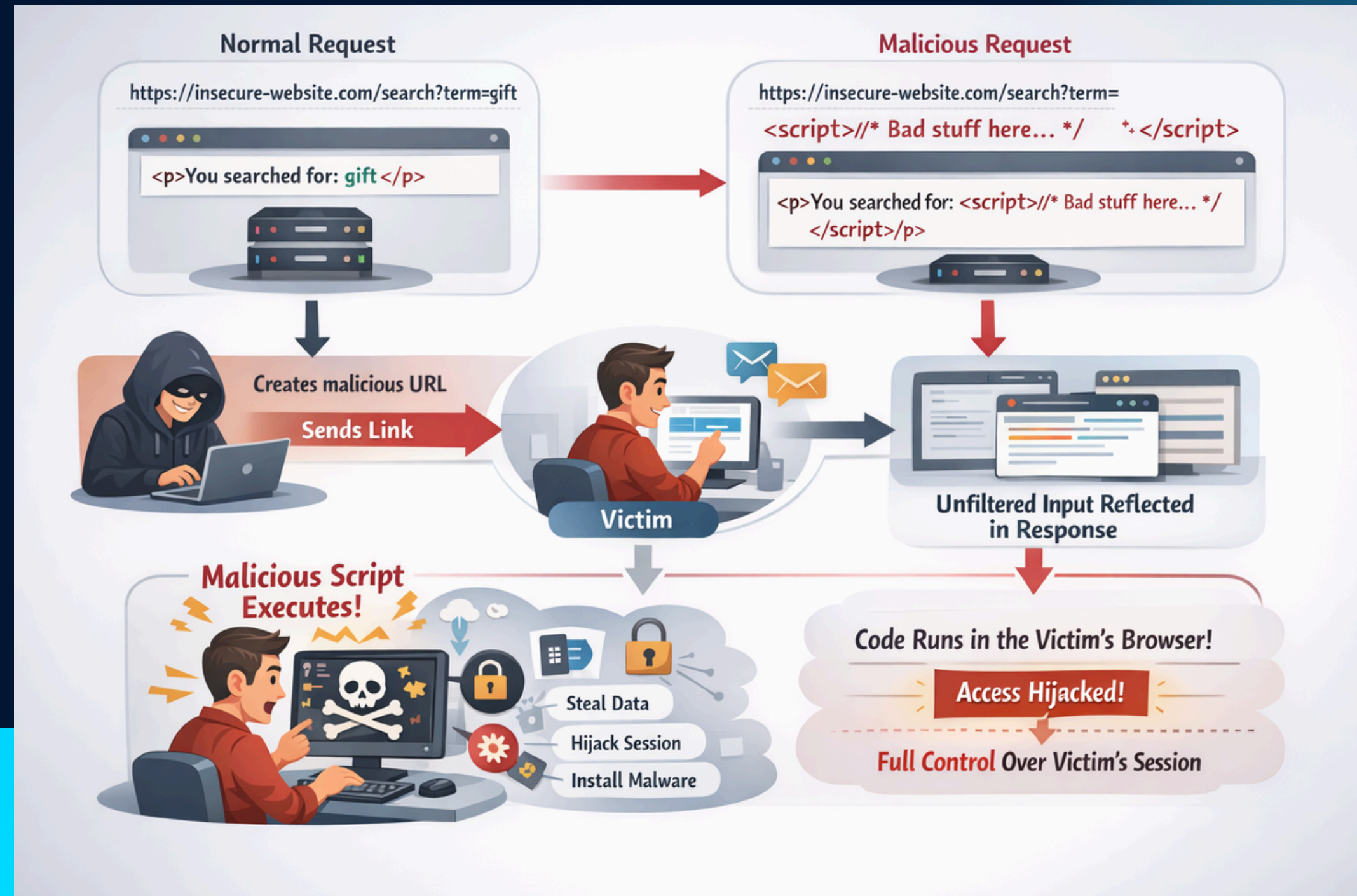
3. Gunakan Content Security Policy (CSP)
Batasi sumber konten yang boleh dijalankan di browser.



4. Lindungi cookie dengan HttpOnly
Cookie dengan HttpOnly tidak dapat diakses oleh JavaScript.

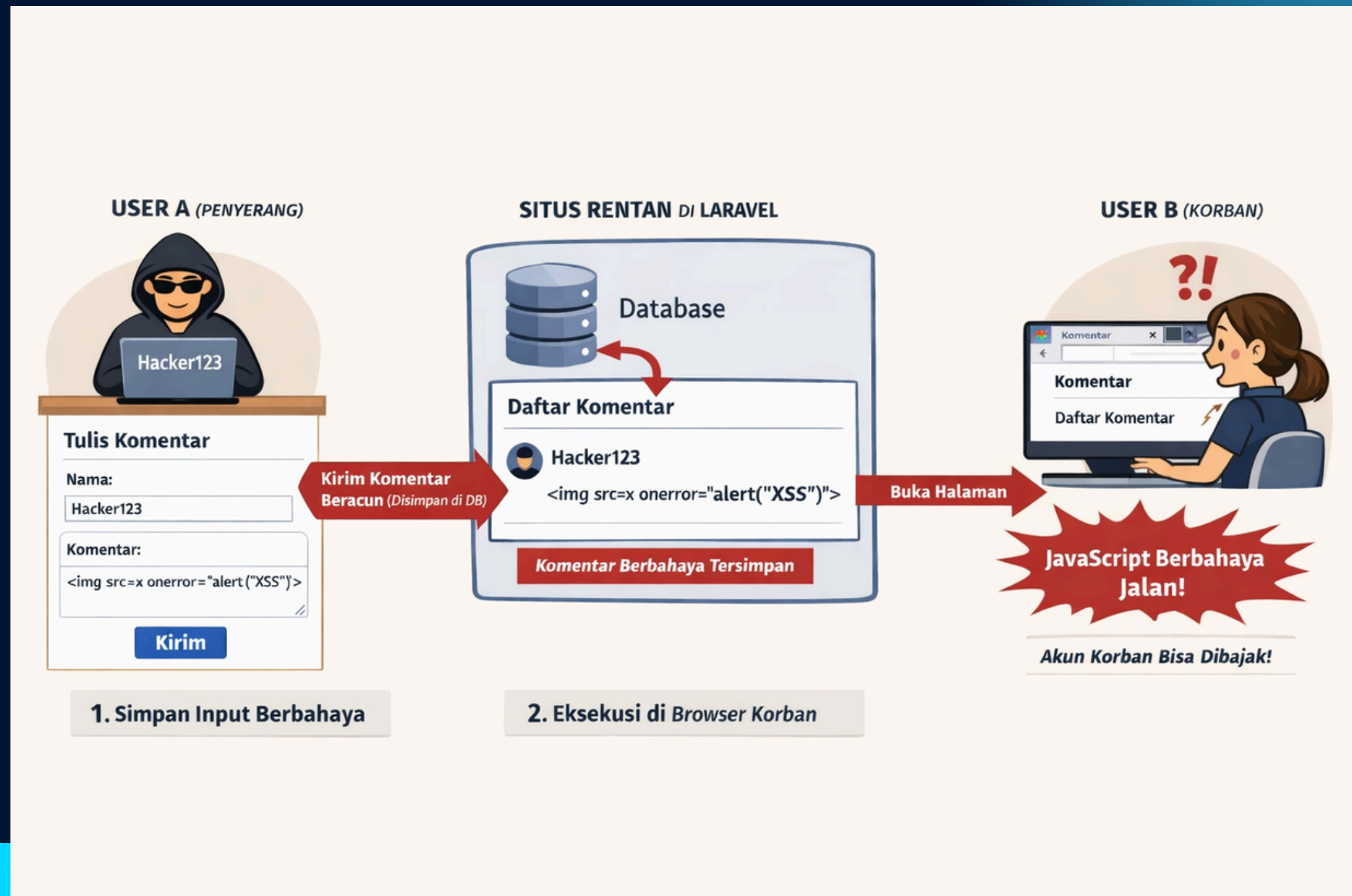
REFLECTED XSS

erjadi ketika sebuah aplikasi menerima data dari HTTP request, lalu menyertakan kembali data tersebut secara langsung ke dalam respons dengan cara yang tidak aman.



STORED XSS

terjadi ketika sebuah aplikasi menerima data dari sumber yang tidak tepercaya, lalu menyimpan data tersebut dan menampilkannya kembali di respons HTTP berikutnya dengan cara yang tidak aman.



DOM-BASED XSS

terjadi ketika sebuah aplikasi menerima data dari sumber yang tidak tepercaya, lalu menyimpan data tersebut dan menampilkannya kembali di respons HTTP berikutnya dengan cara yang tidak aman.

Skenario Teknis DOM-based XSS

Contoh kasus: fitur pencarian membaca input dari URL hash lalu menuliskannya ke halaman dengan cara yang tidak aman.



Alur Teknis Inti

- Source: location.hash / query string / DOM
- Processing: JavaScript membaca dan memproses input
- Sink: innerHTML, document.write, eval, outerHTML
- Trigger: halaman dimuat atau event tertentu dijalankan

Mengapa Rentan?

- Input tidak divalidasi
- Output tidak di-encode
- Penggunaan sink berbahaya
- Logika front-end terlalu percaya pada data dari browser

Perbaikan yang Aman

- ```
const q = location.hash.slice(1);
document.getElementById('result').textContent = q;
```
- Validasi dan sanitasi input
  - Gunakan textContent / innerText untuk teks biasa
  - Hindari innerHTML bila tidak perlu
  - Tambahkan Content Security Policy (CSP)



Inti DOM-based XSS: data dari URL atau DOM diproses oleh JavaScript sisi klien lalu dimasukkan kembali ke halaman tanpa pengamanan yang tepat.

## MENCEGAH XSS

terjadi ketika sebuah aplikasi menerima data dari sumber yang tidak tepercaya, lalu menyimpan data tersebut dan menampilkannya kembali di respons HTTP berikutnya dengan cara yang tidak aman.